

Motivation

Brown Space Engineering (BSE) aims to send the Perovskite Visual Degradation eXperiment (PVDX) satellite to NASA as part of NASA's CubeSat Launch Initiative in January 2027. One of the core components of BSE's work and satellite communications in general is having a way to reliably and quickly send files to and from the satellite. Space is not an easy domain to work in, as there is a limited time of communication between the satellite and ground station, as the average Low-Earth Orbit satellite moves at around 7.8 km/s and thus will not be in range of a ground station for long. Additionally, the long distance that information has to travel over to reach either side of the communication means high latency and a low signal-to-noise ratio. Most satellites use the CCSDS File Delivery Protocol as a way to reliably and efficiently send files to and from a satellite on top of other protocols that are part of the CCSDS. BSE aims to use CFDP as a way of sending files to and from the satellite, as some of the main goals of the satellite involve sending bitmaps/images to and from the satellite, which can be relatively large and thus require the reliable and efficient transfer that CFDP provides.

Goals

Our goals include the following:

- Develop a base CFDP application that BSE can build off of, test, and eventually use for their satellite communications
- Learn more about CFDP and the broader context of satellite communications
- Gain experience with memory-unsafe programming for space applications

What did we achieve?

We achieved a base CFDP application, including the following components:

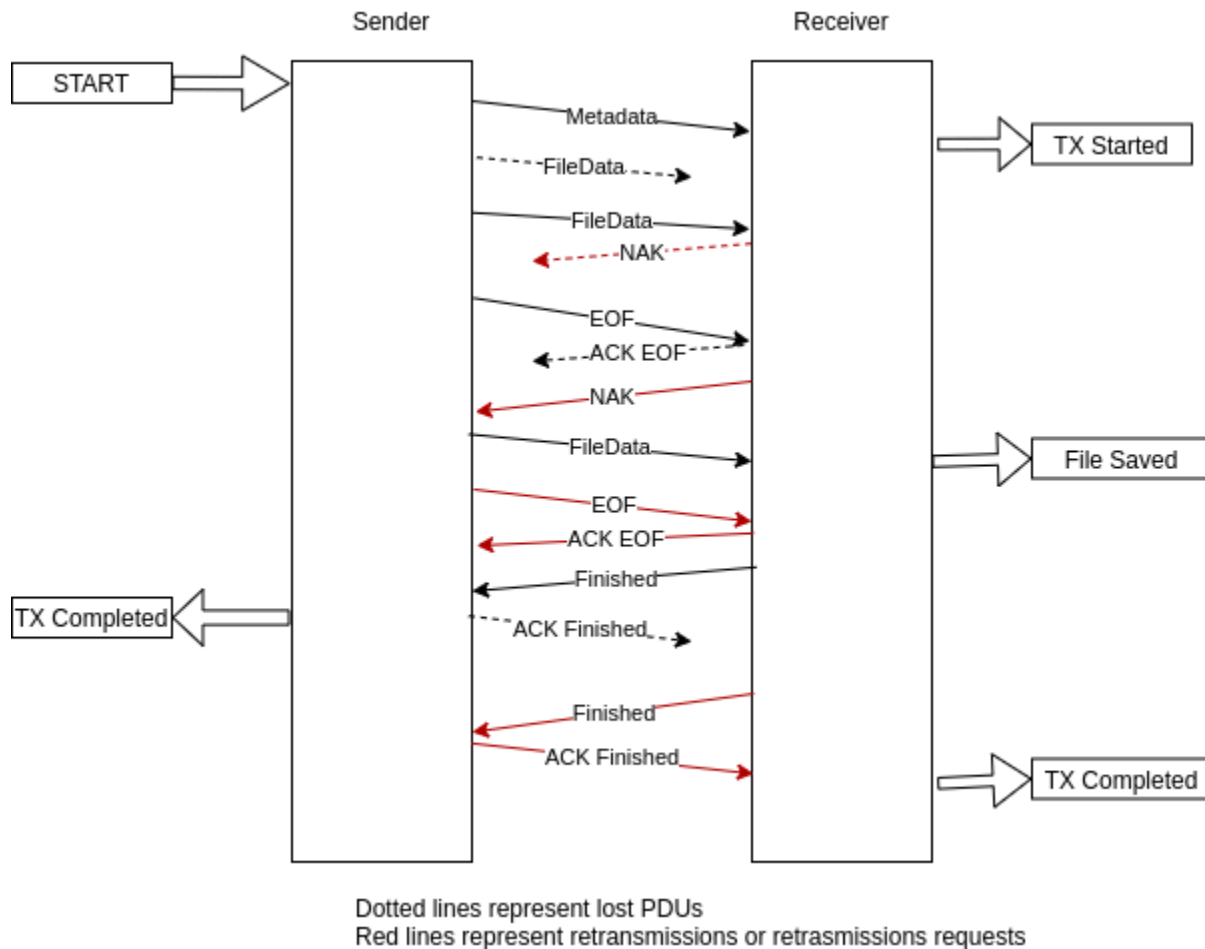
- PDU serialization/deserialization for various types, such as NAK, EOF, Metadata, and Data PDUs
- Class 1 (unacknowledged) sending/receiving (this essentially boils down to UDP with some extra steps)
- 2 modes of checksum verification (modular and additive)
- a simple digital link (UDP based) to test the protocol on

We also achieved Class 2 (acknowledged) sending/receiving, which handles dropped packets using deferred NAKs.

Due to time constraints and relative unimportance, we were unable to implement the following:

- Config reading for certain protocol constants (in the end, this is only going to be used for one application so there isn't a big concern about being modular for different uses)
- Filestore abstraction layer (to stick to the CFDP protocol we will eventually implement this but isn't necessary right now as we demonstrate CFDP file transfer through our digital link)
- Immediate NAKs (these can be useful in certain situations but for high latency, deferred NAKs are way better since congestion control is super important for satellite communications)
- Concurrency, Keep-Alive PDUs, inactivity timers

Design/Implementation



Our implementation follows this figure. We first send metadata about the file we are about to send. This metadata includes information such as the file size, checksum type, and the src/dest filenames. Then the file data itself is sent to the receiver in packets. While this sending is occurring, the receiver collects information about the dropped packets but doesn't send NAKs yet. Once the sender sends an EOF PDU, the receiver then sends back one cumulative NAK for all the missing packets. These are deferred NAKs, and are primarily used over immediate NAKs (NAK sent immediately after detection of dropped packet) because they prevent congestion, which is extremely important for satellite communication where there is high latency. Eventually, the receiver will send back a Finished PDU to let the sender know that it has received all the packets, and a close down occurs.

Discussion/Results

We implemented and tested in steps.

First, we made sure that PDU roundtrips sent the data accurately and reliably (all fields stayed the same).

```
csl680-user@83b173d69dfb:~/final-cfdp$ make test
./bin/test_pdu
=== CFDP PDU Unit Tests ===

-- Header round-trip --
PASS: header encode returns positive length
PASS: header decode consumes same bytes as encode
PASS: version
PASS: pdu_type
PASS: direction
PASS: tx_mode
PASS: data_field_len
PASS: entity_id_len
PASS: seq_num_len
PASS: source_entity_id
PASS: dest_entity_id
PASS: transaction_seq_num

-- Metadata round-trip --
PASS: metadata encode returns positive length
PASS: metadata decode consumes same bytes as encode
PASS: closure_requested
PASS: checksum_type
PASS: file_size
PASS: src_filename
PASS: dst_filename

-- File Data round-trip --
PASS: file_data encode length
PASS: file_data decode consumes all bytes
PASS: offset
PASS: data_len
PASS: payload bytes

-- EOF round-trip --
PASS: eof encode length == 10
PASS: eof decode consumes all bytes
PASS: condition_code
PASS: checksum
PASS: eof_file_size

-- Checksum --
PASS: modular checksum known value
PASS: incremental checksum matches full checksum

-- Edge cases --
PASS: eof encode into tiny buffer errors
PASS: truncated header decode fails cleanly

=====
All tests PASSED.
```

We then went on to test Class 1 sending/receiving under 0 loss.

```
[SENDER] Transfer complete. State: FINISHED
[RECV] Got File Data: offset=1253376 len=512 (1253888 bytes total)
[RECV] Got File Data: offset=1253888 len=58 (1253946 bytes total)
[RECV] Got EOF: condition=0, checksum=0xB0B5942C, size=1253946
[RECV] Checksum OK (0xB0B5942C). Transfer complete.
[RECV] File written to 'received_file.bin' (1253946 bytes)

[RECV] Transfer complete. State: FINISHED

=== LOOPBACK TEST PASSED: files match ===
```

We also wanted to make sure that sending and receiving in Class 1 under >0 loss would fail, as there was no retransmission method in place yet to prevent dropped packets from being lost.

```
[TRANSPORT] *** DROPPED PDU (17 bytes) ***
[SENDER] Sent EOF: checksum=0xB0B5942C, file_size=1253946

[SENDER] Transfer complete. State: FINISHED
[RECV] Got File Data: offset=1253888 len=58      (1075258 bytes total)

=== LOOPBACK TEST FAILED ===
```

Finally, we tested Class 2 sending/receiving, and found that we were able to retransmit and eventually get back the full file under a loss of 5.

```
[RECV2] Check timer expired - resending NAK (retry 3/3)
[RECV2] Sent NAK: 2 gap(s)
[RECV2]   gap 0: [1197568, 1198080)
[RECV2]   gap 1: [1236992, 1237504)
[SENDER2] Received NAK: 2 gap(s)
[SENDER2]   gap [1197568, 1198080)
[SENDER2]   gap [1236992, 1237504)
[SENDER2] Retransmit: offset=1197568 len=512      (gap 0 [1197568,1198080))
[SENDER2] Retransmit: offset=1236992 len=512      (gap 1 [1236992,1237504))
[RECV2] Got File Data: offset=1197568 len=512      gaps=1
[RECV2] Got File Data: offset=1236992 len=512      gaps=0
[RECV2] Checksum OK (0xB0B5942C). Sending Finished.
[RECV2] Sent Finished: delivery=COMPLETE cond=0
[SENDER2] Received Finished: delivery=COMPLETE
[SENDER2] Sent ACK(Finished)

[SENDER2] Transfer complete. State: FINISHED
[RECV2] Got ACK(Finished). Transfer complete.
[RECV2] File written to 'received_file.bin' (1253946 bytes)

[RECEIVER2] Transfer complete. State: FINISHED

=== CLASS 2 LOOPBACK TEST PASSED: files match ===
```

Under larger amounts of loss (15% drop rate), the implementation failed to work due to timers taking too long. We weren't able to fix this in time, but it is something we can work on in the future.

```
[RECV2] Check timer limit reached - CANCELLED
Receiver run failed: ERR_CHECKSUM
[SENDER2] Inactivity timeout waiting for Finished - CANCELLED
Sender run failed: ERR_TRANSPORT

=== CLASS 2 LOOPBACK TEST FAILED ===
```

AI usage and experience

We used Claude Sonnet 4.6 to generate tests and util files since those would have taken several hours of tedious work, but we implemented the metadata packets, ACK/NAK messages, and closing handshake ourselves, since those were crucial for the space mission and thus could not afford to have bugs. We verified Claude's output by stopping at checkpoints to double-check the work; for the most part, Claude wrote correct code on its first try (except for the Makefile, which we manually fixed). We also used Claude to debug the code that we had written manually; verifying accuracy of code was simpler than, say, TCP, as we could just verify the gaps, lengths, and offsets sent and received and then also check the full received file against the original to determine correctness.

Overall, we were shocked by how competent the Claude model was, especially since it wasn't the paid subscription version. For future development, we would like to use Claude to debug the tests failing for higher-loss transmissions.

Conclusions/Future Work

We have a working base implementation that BSE can use to test and improve upon for their communications stack. We learned about how CFDP works, especially about the complex checksum calculations, how NAKs work, and the different types of PDUs involved. We also learned about the larger context of satellite communications and how different protocols interacted with each other (there exist other protocols under CCSDS such as USLP and COP-1). We also got experience writing and reading memory unsafe code.

We feel good about this project because it has a clear use case and managed to work for the most part. If we could keep working on it, we would continue AI-assisted debugging, and we would also implement some of the things we weren't able to because of time constraints, such as the filestore abstraction and Keep-Alive PDUs.